

CSCI-GA.1170-001 Fundamental Algorithms

September 8, 2026

Problem Set 1

Lecturer: Julia Kempe

Due: 11:59pm on Monday, September 14

Problem 1-1 (Asymptotic Comparisons)**12 points**

For each of the following pairs of functions $f(n)$ and $g(n)$, state whether f is $O(g)$; whether f is $o(g)$; whether f is $\Theta(g)$; whether f is $\Omega(g)$; and whether f is $\omega(g)$. (More than one of these can be true for a single pair!)

- (a) $f(n) = 3n^8 + 20n^6$, $g(n) = 2n^8 - 300n^6$
- (b) $f(n) = (\log_{10} n)^3$, $g(n) = \log_2(n^3) + (\log_{10} n)$.
- (c) $f(n) = n^n$, $g(n) = 2^{n^2}$.
- (d) $f(n) = n^2(\log_2 n)$, $g(n) = n(\log_2 n)^5$.
- (e) $f(n) = 25^n \cdot (n + \log_2 n)^3$, $g(n) = 5^{2n} \cdot \sqrt{n^6 - n^5}$.
- (f) $f(n) = \log_2 n$, $g(n) = n^\epsilon (\forall \epsilon > 0)$.

Problem 1-2 (Order of Growth)**12 points**

- (a) **(8 points)** For each of the following functions $f(n)$, find a “canonical” function¹ $g(n)$ such that $f(n) = \Theta(g(n))$. For example,

$$3200n + 2n^2 \log_2^4(n^2) = \Theta(n^2 \log_2^4 n).$$

Briefly justify your answers (keep it to 1-2 sentences).

$3^n \cdot (\log_3(n^2))^5$, $(n \cdot \log_2(n^2))^3$, $2^{2n} \cdot \log_2(n^2 + n)$, $n^2 \cdot \sqrt{n} + 1$, $\sqrt{\log_4(n^4)}$, $n^{\frac{n}{\log_2 n}} \cdot \log_{n^4}(2n)$, $n + \log_n(4)$, $n \cdot \log_2(n!)$

- (b) **(4 points)** Based on your answers in part (a), sort the resulting “canonical” (not original)² functions in asymptotically increasing order.

¹i.e., function of the form $a^n n^b (\log n)^c$ for constants $a \geq 1$ and $b, c \geq 0$.

²This way even if you got part (a) wrong, you can still have correct solution to part (b).

Problem 1-3 (Counting Points)**16(+3) points**

You are given a set of n points $x_1, \dots, x_n$ and a set of n segments $[a_1, b_1], \dots, [a_n, b_n]$ on a line, where x_i, a_i, b_i are all integers. We want to output $k_1, k_2, \dots, k_n$, where k_i is the number of *points* contained in the i -th *segment*. More formally, let $X[1, \dots, n]$ be the array storing the points. Moreover, let $I[1, \dots, n]$ denote an array storing the segments such that each entry $I[i]$ stores two values $I[i].left$ and $I[i].right$. We want to write an algorithm that computes $K[1, \dots, n]$ with

$$K[i] = |\{j : I[i].left \leq X[j] \leq I[i].right\}|,$$

i.e. $K[i]$ is the number of points contained in the segment $I[i]$. In parts (a)–(e), assume that all the points, starting points, and endpoints are distinct.

- (a) (1 point) Assume you solve the problem using a naive algorithm which iterates through all point-segment pairs, what is the running time?

In the following, we develop a more clever algorithm. To this end, we first create an auxiliary array $C[1, \dots, 3n]$ that stores the points as well as start and end points of the intervals. Each entry $C[j]$ stores three values: $C[j].value$ that is an integer, $C[j].type$ that is either of the literals “left”, “right”, or “point”, and a value $C[j].pointer$ that stores an index between 1 and n into one of the original arrays.

- (b) (3 points) Write the pseudocode of a procedure `GENERATEC( $X, I, n$ )` that fills the above described array C based on the arrays X and I .
- (c) (4 points) Assume now that the above described array C has been generated and then sorted according to the value field, i.e., such that $C[i].value < C[i+1].value$ for all i from 1 to $3n-1$. Write a procedure `COMPUTEK( $C, n$ )` which computes the array $K[1, \dots, n]$ in time $\Theta(n)$ by iterating over the array C *once*, using only $O(1)$ additional memory beyond the output array K . In particular, you may not maintain another array of the currently open segments (If you give the approach using additional array, you will receive partial marks).

Hint: a single counter suffices if you are willing to let $K[i]$ hold a temporary, possibly negative, intermediate value.

- (d) (2 point) Assume you use the best sorting algorithm you know from the lecture so far, what is the overall running time when combining the solutions from parts (b) and (c) to solve the overall problem?
- (e) (4 points) Argue the correctness of part (c) using a suitable loop invariant.
- (f) (2 points) Now assume that collisions between points and start or endpoints of the intervals can happen, as well as collisions between multiple intervals. For instance, we could have that $X[i] = I[j].left$, that $I[i].left = I[j].right$, or that $I[i].left = I[j].left$ for some i and j . Show that if we do not make any assumptions on how the sorting algorithm breaks ties, that then our algorithm is incorrect. To this end, provide an example input X and I and a sorted corresponding C such that your algorithm from part (c) produces the wrong result.
- (g) (Extra Credit) (3 points) Generalize your algorithm from part (b) in such a way that the combined algorithm, with unmodified part (c), correctly works even in the presence of collisions. The format of the array C should remain unchanged and we still do not make any assumption on how the sorting algorithm breaks ties.

Hint: You may consider tweaking the values stored in $C[i].value$.